

Tarea 2 — Constraint Satisfaction Problem (CSP)

Asignación de Microservicios a Servidores Físicos

- Ricardo Godinez 23247
- Vianka Castro 23201

Link al repo: https://github.com/Vann06/Inteligencia_Artificial/tree/LAB8

Problema:

- **Variables:** 8 microservicios M1 ... M8
- **Dominio:** {S1, S2, S3} — servidor asignado a cada microservicio
- **Restricciones:**
 1. **Capacidad (global):** ningún servidor puede alojar más de 3 microservicios
 2. **Anti-afinidad (binaria):** los pares (M1,M2), (M3,M4), (M5,M6), (M1,M5) no pueden compartir servidor

| Task | Algoritmo | ||||| | 2.1 | Backtracking Search con Forward Checking (Lookahead) | | 2.2 | Beam Search con parámetro K configurable | | 2.3 | Local Search — Modos Condicionales Iterados (ICM) | | 2.4 | Benchmarking y Conclusiones |

Definición Compartida del CSP

Estas constantes son utilizadas por los tres algoritmos.

Estructura del notebook (codebase)

- **Definición del CSP:** variables, dominio, capacidad y pares de anti-afinidad (y el grafo `CONFLICTS`).
- **Task 2.1:** Backtracking + Forward Checking (con MRV/LCV) y validación de la solución.
- **Task 2.2:** Beam Search con peso heurístico y `K` configurable.
- **Task 2.3:** ICM (búsqueda local) con reinicios aleatorios.
- **Task 2.4:** benchmarking para comparar éxito/tiempo/operaciones.

```
In [ ]: import time
import random
from copy import deepcopy
from dataclasses import dataclass, field
```

```

from typing import Optional

VARIABLES = ['M1', 'M2', 'M3', 'M4', 'M5', 'M6', 'M7', 'M8']
SERVERS = ['S1', 'S2', 'S3']
MAX_CAPACITY = 3

ANTI_AFFINITY_PAIRS = [
    ('M1', 'M2'), # BD primaria y réplica
    ('M3', 'M4'),
    ('M5', 'M6'),
    ('M1', 'M5'),
]

# Grafo de conflictos para búsqueda 0(1)
CONFLICTS: dict[str, set[str]] = {v: set() for v in VARIABLES}
for _a, _b in ANTI_AFFINITY_PAIRS:
    CONFLICTS[_a].add(_b)
    CONFLICTS[_b].add(_a)

```

Task 2.1 — Backtracking Search con Forward Checking

Algoritmo:

1. Seleccionar variable no asignada (heurística **MRV** — Minimum Remaining Values).
2. Para cada valor en su dominio actual:
 - Verificar consistencia con la asignación actual (`is_consistent`).
 - Asignar y aplicar **Forward Checking**: podar dominios de variables futuras.
 - Si algún dominio queda vacío → *wipe-out* → retroceder (**backtrack**).
3. Repetir hasta asignación completa o agotar opciones.

Propiedades: Completo . Óptimo (encuentra solución si existe) . Exponencial en peor caso.

```

In [ ]: # =====
# BACKTRACKING SEARCH – Backtrack(x, w, Domains)
# x      : asignación parcial
# w      : peso acumulado (producto de factores  $\delta$ )
# Domains : dominios actuales (podados por FC)
# =====

def is_consistent(var: str, value: str, assignment: dict) -> bool:
    for neighbor in CONFLICTS[var]:
        if neighbor in assignment and assignment[neighbor] == value:
            return False # f_j anti-afinidad = 0 →  $\delta = 0$ 
    current_load = sum(1 for s in assignment.values() if s == value)
    if current_load >= MAX_CAPACITY:
        return False # f_j capacidad = 0 →  $\delta = 0$ 
    return True #  $\delta > 0$ , valor candidato válido

```

```

# "Domains' ← Domain via LOOKAHEAD"
# Si algún Domains'_i queda vacío → "Si cualquier Domains'_i está vacío: Cont
def forward_check(var: str, value: str, assignment: dict, domains: dict) ->
    new_domains = {v: list(d) for v, d in domains.items()}
    server_load = {s: 0 for s in SERVERS}
    for assigned_server in assignment.values():
        server_load[assigned_server] += 1

    for unassigned in VARIABLES:
        if unassigned in assignment:
            continue
        to_remove = []
        for server in new_domains[unassigned]:
            pruned = False
            if unassigned in CONFLICTS[server] and server == value:
                to_remove.append(server); pruned = True
            if not pruned and server_load[server] >= MAX_CAPACITY:
                to_remove.append(server)
        for s in to_remove:
            if s in new_domains[unassigned]:
                new_domains[unassigned].remove(s)
        if not new_domains[unassigned]:
            return None # Domains'_i vacío → podar rama
    return new_domains

# -- 2.1.3 Selección de Variable (MRV) -----
def select_unassigned_variable(assignment: dict, domains: dict) -> str:
    unassigned = [v for v in VARIABLES if v not in assignment]
    return min(unassigned, key=lambda v: len(domains[v]))

# "Ordenar los VALORES del Domain_i de la Xi elegida" – heurística LCV
def order_values(var: str, domains: dict, assignment: dict) -> list:
    def lcv_score(value):
        removed = 0
        for neighbor in CONFLICTS[var]:
            if neighbor not in assignment and value in domains[neighbor]:
                removed += 1
        return removed
    return sorted(domains[var], key=lcv_score)

class BT_Stats:
    nodes = backtracks = fc_prunes = 0

# Implementación de Backtrack(x, w, Domains):
# 1. Asignación completa → retornar solución
# 2. Elegir Xi (MRV) → select_unassigned_variable
# 3. Ordenar valores (LCV) → order_values
# 4.  $\delta \leftarrow \prod f_j(\dots)$  → is_consistent (Si  $\delta=0$ : continuar)
# 5. Domains' via LOOKAHEAD → forward_check (Si vacío: continuar)
# 6. Backtrack( $x \cup \{X_i:v\}$ ,  $w\delta$ , Domains') → llamada recursiva
def backtrack(assignment: dict, domains: dict, stats: BT_Stats) -> dict | No

```

```

if len(assignment) == len(VARIABLES):  # paso 1: asignación completa
    return assignment

var = select_unassigned_variable(assignment, domains)  # paso 2

for value in order_values(var, domains, assignment):  # paso 3
    stats.nodes += 1

    if is_consistent(var, value, assignment):          # paso 4:  $\delta > 0$ 
                                                         #  $x \leftarrow x \cup \{X_i:v\}$ 
        assignment[var] = value

        pruned_domains = forward_check(var, value, assignment, domains)

        if pruned_domains is not None:
            result = backtrack(assignment, pruned_domains, stats)
            if result is not None:
                return result
        else:
            stats.fc_prunes += 1

    del assignment[var]  # deshacer:  $x \leftarrow x \setminus \{X_i\}$ 
    stats.backtracks += 1

return None

# Confirma que  $\delta > 0$  para todos los  $f_j$  en la solución final.
def bt_validate(solution: dict) -> bool:
    valid = True
    for a, b in ANTI_AFFINITY_PAIRS:
        if solution[a] == solution[b]:
            valid = False; print(f" [FAIL] Anti-afinidad violada: {a}={sol
        else:
            print(f" [OK] {a}={solution[a]} != {b}={solution[b]}")
    for s in SERVERS:
        micros = [v for v in VARIABLES if solution[v] == s]
        ok = len(micros) <= MAX_CAPACITY
        icon = "[OK]" if ok else "[FAIL]"
        if not ok: valid = False
        print(f" {icon} {s}: {len(micros)} microservicios {micros}")
    return valid

```

Ejecución (Task 2.1)

En esta celda se corre Backtracking con MRV + Forward Checking para encontrar una asignación válida.

Observaciones:

- El tiempo y las estadísticas (nodos/backtracks/podas).
- La distribución por servidor (nadie debe pasar de 3 microservicios).
- La validación: todos los pares anti-afinidad deben quedar en servidores distintos.

```

In [ ]: initial_domains = {v: list(SERVERS) for v in VARIABLES}
        bt_stats       = BT_Stats()

t0      = time.perf_counter()
bt_solution = backtrack({}, initial_domains, bt_stats)
bt_elapsed = (time.perf_counter() - t0) * 1000

if bt_solution:
    print(f"Nodos expandidos : {bt_stats.nodes}")
    print(f"Backtracks      : {bt_stats.backtracks}")
    print(f"Podas por FC     : {bt_stats.fc_prunes}")
    print(f"Tiempo (ms)      : {bt_elapsed:.4f}")

    server_map = {s: [] for s in SERVERS}
    for micro, srv in sorted(bt_solution.items()):
        server_map[srv].append(micro)

    print("\nDistribución:")
    for srv, micros in server_map.items():
        bar = "#"*len(micros) + "."*(MAX_CAPACITY - len(micros))
        print(f" {srv} [{len(micros)}/{MAX_CAPACITY}] {bar} -> {'', '.jo

    print(f"\nAsignación: {dict(sorted(bt_solution.items()))}")
    ok = bt_validate(bt_solution)
    print(f"\nVálida: {'Sí' if ok else 'No'}")
else:
    print("No se encontró solución.")

```

```

Nodos expandidos : 8
Backtracks      : 0
Podas por FC     : 0
Tiempo (ms)      : 0.1393

```

```

Distribución:
S1 [3/3] ### -> M1, M3, M6
S2 [3/3] ### -> M2, M4, M5
S3 [2/3] ##. -> M7, M8

```

```

Asignación: {'M1': 'S1', 'M2': 'S2', 'M3': 'S1', 'M4': 'S2', 'M5': 'S2', 'M6': 'S1', 'M7': 'S3', 'M8': 'S3'}
[OK] M1=S1 != M2=S2
[OK] M3=S1 != M4=S2
[OK] M5=S2 != M6=S1
[OK] M1=S1 != M5=S2
[OK] S1: 3 microservicios ['M1', 'M3', 'M6']
[OK] S2: 3 microservicios ['M2', 'M4', 'M5']
[OK] S3: 2 microservicios ['M7', 'M8']

```

Válida: Sí

Resultados

En la corrida mostrada se encontró una asignación válida y la carga quedó bien repartida (3–3–2). La validación confirma que se cumplen capacidad y anti-afinidad.

Además se reportan estadísticas de nodos explorados, backtracks y podas por Forward Checking. podemos notar que el algoritmo es eficiente para este problema específico, encontrando una solución sin necesidad de retroceder.

Task 2.2 — Beam Search (K configurable)

Algoritmo (conforme a diapositiva 7-9):

```
Init    C = [{}]  
Para i = 1,...,n:  
    Extend : C' <- { x U {Xi:v} : x in C, v in Domaini }  
    Prune  : C <- K elementos de C' con los PESOS MÁS  
    GRANDES
```

Función de peso: $\text{weight}(x) = -(\alpha \cdot \text{anti_afinidad} + \beta \cdot \text{capacidad} + \gamma \cdot \text{riesgo_futuro})$ donde $\alpha=10, \beta=20, \gamma=1 \rightarrow$ **mayor peso = mejor candidato.**

Propiedades: Incompleto . No garantiza solución . Configurable con K . Rápido.

```
In [ ]: # =====  
# BEAM SEARCH – Mantener ≤ K candidatos C de asignaciones parciales  
# C : lista de asignaciones parciales (el "beam")  
# K : ancho máximo del beam  
# w : peso de cada candidato → C ← K elementos con pesos más grandes  
# =====  
  
# El peso w(x) determina qué K candidatos se retienen en Prune.  
# Mayor peso = menos penalización = mejor candidato.  
W_ANTI_AFFINITY = 10 # penalización por par en mismo servidor  
W_CAPACITY_OVER = 20 # penalización por exceder capacidad  
W_FUTURE_RISK   = 1  # penalización anticipada por riesgo futuro  
  
# Define el criterio de "pesos más grandes" del paso Prune.  
# weight(x) = -(penalización total) → mayor peso = mejor candidato  
# weight = 0 significa asignación perfecta sin violaciones.  
def candidate_weight(assignment: dict) -> tuple[int, int]:  
    hard_penalty = 0  
    future_risk  = 0  
  
    # Penalizar violaciones de anti-afinidad ya cometidas  
    for a, b in ANTI_AFFINITY_PAIRS:  
        if a in assignment and b in assignment:  
            if assignment[a] == assignment[b]:  
                hard_penalty += W_ANTI_AFFINITY  
  
    # Penalizar servidores que exceden capacidad máxima  
    server_load = {s: 0 for s in SERVERS}  
    for srv in assignment.values():
```

```

        server_load[srv] += 1
    for srv, load in server_load.items():
        if load > MAX_CAPACITY:
            hard_penalty += W_CAPACITY_OVER * (load - MAX_CAPACITY)

    # Riesgo futuro: vecino no asignado con pocas opciones válidas
    unassigned = [v for v in VARIABLES if v not in assignment]
    for a, b in ANTI_AFFINITY_PAIRS:
        if (a in assignment) ^ (b in assignment):
            free_var = b if a in assignment else a
            fixed_var = a if a in assignment else b
            fixed_srv = assignment[fixed_var]
            if free_var in unassigned:
                options = sum(
                    1 for s in SERVERS
                    if s != fixed_srv and server_load[s] < MAX_CAPACITY
                )
                future_risk += W_FUTURE_RISK * (len(SERVERS) - options)

    total_penalty = hard_penalty + future_risk
    return -total_penalty, hard_penalty # (weight, violaciones hard)

def count_hard_violations_bs(assignment: dict) -> int:
    _, hard = candidate_weight(assignment)
    return hard

# Cada BeamState es un candidato  $x \in C$  con su peso asociado.
# El campo weight define el orden para el paso Prune.
@dataclass(order=True)
class BeamState:
    weight : int = field(compare=True) # criterio de Prune: pesos más
    hard_viol : int = field(compare=False)
    assignment: dict = field(compare=False)

    def __repr__(self):
        asgn = {k: self.assignment[k] for k in VARIABLES if k in self.assignment}
        return f"BS(w={self.weight}, viol={self.hard_viol}, {asgn})"

@dataclass
class BeamStats:
    nodes_generated: int = 0
    nodes_pruned : int = 0
    beam_per_level : list[int] = field(default_factory=list)
    solution_found : bool = False

# Implementación directa del algoritmo de la diapositiva:
#
# Init  $C = [\{\}]$  → beam = [BeamState({})]
# Para  $i = 1, \dots, n$ :
# Extend:  $C' \leftarrow \{x \cup \{X_i:v\} : x \in C, v \in \text{Domain}_i\}$  → bucle sobre beam  $\times$ 
# Prune:  $C \leftarrow K$  elementos de  $C'$  con pesos más grandes → sort + slice [
#

```

```

def beam_search(K: int, verbose: bool = False) -> tuple[Optional[dict], BeamStats]
    stats = BeamStats()

    # Init C = [{}]
    beam = [BeamState(weight=0, hard_viol=0, assignment={})]

    if verbose:
        print(f" Nivel  0 | Init C = [{{}}] (K={K})")

    for level, var in enumerate(VARIABLES):
        successors = []

        # EXTEND: C' ← {x ∪ {Xi:v} : x ∈ C, v ∈ Domaini}
        for state in beam:
            for server in SERVERS:
                new_asgn = dict(state.assignment)
                new_asgn[var] = server # x ∪ {Xi:v}
                w, hard = candidate_weight(new_asgn)
                successors.append(BeamState(weight=w, hard_viol=hard, assignment=new_asgn))
                stats.nodes_generated += 1

        if not successors:
            break

        # PRUNE: C ← K elementos de C' con los pesos más grandes
        successors.sort(reverse=True) # mayor weight primero
        pruned = len(successors) - K
        beam = successors[:K] # conservar solo K mejores
        stats.nodes_pruned += max(0, pruned)
        stats.beam_per_level.append(len(beam))

        if verbose:
            print(f" Nivel {level+1:2d} | var={var} generados={stats.nodes_generated}")
            for i, s in enumerate(beam[:3], 1):
                asgn = {k: s.assignment[k] for k in VARIABLES if k in s.assignment}
                print(f"   #{i} weight={s.weight:4d} viol={s.hard_viol}")
            if len(beam) > 3:
                print(f"   ... ({len(beam)-3} más)")

        # Verificar si el beam final contiene una solución válida
        for state in beam:
            if len(state.assignment) == len(VARIABLES) and count_hard_violations(state) == 0:
                stats.solution_found = True
                return state.assignment, stats

    return None, stats # NOTA: puede no encontrar solución (no es completo)

```

Observaciones

Beam Search mantiene solo los mejores K candidatos por nivel. Por eso es rápido, pero con K pequeño puede descartar la rama que llevaba a la solución. Por lo tanto es importante experimentar con diferentes valores de K para balancear entre eficiencia y probabilidad de encontrar una solución. Y que el peso heurístico esté bien diseñado para guiar la búsqueda hacia soluciones prometedoras.

En la siguiente celda:

- Se muestra una corrida detallada con `K=3`.
- Luego se compara el comportamiento para varios valores de `K`.

```
In [ ]: # Por defecto lo dejamos sin traza (menos texto). Si querés ver el paso a pa
SHOW_TRACE = False

K_DEMO = 3
t0 = time.perf_counter()
bs_sol, bs_st = beam_search(K=K_DEMO, verbose=SHOW_TRACE)
bs_elapsed = (time.perf_counter() - t0) * 1000

if bs_sol:
    server_map = {s: [] for s in SERVERS}
    for micro, srv in sorted(bs_sol.items()):
        server_map[srv].append(micro)
    print(f"Solución encontrada (K={K_DEMO}):")
    for srv, micros in server_map.items():
        bar = "#" * len(micros) + "." * (MAX_CAPACITY - len(micros))
        print(f" {srv} [{len(micros)}/{MAX_CAPACITY}] {bar} -> {'', '}.join
    print(f"Asignación: {dict(sorted(bs_sol.items()))}")
    print(f"Nodos generados: {bs_st.nodes_generated} | Podados: {bs_st.nodes
else:
    print(f"No se encontró solución con K={K_DEMO}.")

# Comparativa de varios K
k_values = [1, 2, 3, 5, 8, 12, 24]
print("\nComparativa (mientras más K, menos riesgo de podar la rama correcta
print(f" { 'K':>4} | { 'Solución':^10} | { 'Generados':>9} | { 'Podados':>7} |
print(" " + "-" * 54)
for K in k_values:
    t0 = time.perf_counter()
    sol, st = beam_search(K=K, verbose=False)
    ms = (time.perf_counter() - t0) * 1000
    found = "Sí" if sol else "No"
    print(f" {K:>4} | {found:^10} | {st.nodes_generated:>9} | {st.nodes_pr
```

Solución encontrada (K=3):

S1 [3/3] ### -> M1, M3, M6

S2 [3/3] ### -> M2, M4, M5

S3 [2/3] ##. -> M7, M8

Asignación: {'M1': 'S1', 'M2': 'S2', 'M3': 'S1', 'M4': 'S2', 'M5': 'S2', 'M6': 'S1', 'M7': 'S3', 'M8': 'S3'}

Nodos generados: 66 | Podados: 42 | Tiempo (ms): 0.4839

Comparativa (mientras más K, menos riesgo de podar la rama correcta):

K	Solución	Generados	Podados	ms
1	Sí	24	16	0.0867
2	Sí	45	29	0.1335
3	Sí	66	42	0.1896
5	Sí	102	64	0.2813
8	Sí	156	97	0.4354
12	Sí	219	135	0.6021
24	Sí	399	243	1.4787

Resultados

Para este CSP, $K=3$ sí encontró solución y la tabla muestra que incluso con K más pequeño también se logra en esta instancia. Lo importante es la idea: si en algún momento falla con K bajo, suele ser por la poda no porque el CSP no tenga solución. Al final lo más importante es notar que el valor de K afecta la eficiencia y la probabilidad de éxito, y que el peso heurístico es clave para guiar la búsqueda hacia soluciones prometedoras.

Task 2.3 — Local Search: ICM (Modos Condicionales Iterados)

Algoritmo (conforme a diapositiva 15):

```

Init  x  a una asignación completa aleatoria
Itere por i = 1,...,n hasta converger:
    Calcule pesos de  xv = x U {Xi : v}  para cada v in
Domaini
    x <- xv  con el peso mayor

```

Función de peso: $\text{weight}(x) = -\text{count_violations}(x) \rightarrow \text{mayor peso} = \text{mejor asignación}.$

Inicio aleatorio $\rightarrow$ muchas violaciones. Cada sweep mejora monótonamente. **Parada:** ninguna variable cambia en un sweep completo (óptimo local) o se alcanza MAX_ITER . **Reinicios aleatorios** para escapar óptimos locales.

Propiedades: Completo con reinicios . Converge siempre . Puede quedar en óptimo local.

```
In [ ]: # =====
# MODOS CONDICIONALES ITERADOS (ICM)
# Algoritmo:
# 1. Init x a una asignación completa aleatoria
# 2. Itere por i = 1,...,n hasta converger:
#     · Calcule pesos de  $xv = x \cup \{Xi:v\}$  para cada v
#     ·  $x \leftarrow xv$  con el peso mayor
# =====

# Base del peso: weight(x) = -count_violations(x)
# Menos violaciones = mayor peso = mejor asignación.
def count_violations(assignment: dict) -> int:
    violations = 0
    for a, b in ANTI_AFFINITY_PAIRS:
        if assignment.get(a) == assignment.get(b):
            violations += 1
    for s in SERVERS:
        load = sum(1 for v in VARIABLES if assignment.get(v) == s)
        if load > MAX_CAPACITY:
            violations += load - MAX_CAPACITY
    return violations

# weight(x) = -count_violations(x) → mayor peso = menos violaciones
# weight = 0 → asignación perfecta, sin violaciones
def assignment_weight(assignment: dict) -> int:
    return -count_violations(assignment)

def violations_breakdown(assignment: dict) -> dict:
    anti = [f"{a}={assignment[a]} == {b}={assignment[b]}"
            for a, b in ANTI_AFFINITY_PAIRS if assignment.get(a) == assignment.get(b)]
    cap = [f"{s}: {sum(1 for v in VARIABLES if assignment.get(v)==s)}/{MAX_CAPACITY}"
            for s in SERVERS
            if sum(1 for v in VARIABLES if assignment.get(v)==s) > MAX_CAPACITY]
    return {"anti_affinity": anti, "capacity": cap}

# El punto de inicio es aleatorio y probablemente tiene violaciones.
def random_assignment(seed=None) -> dict:
    rng = random.Random(seed)
    return {v: rng.choice(SERVERS) for v in VARIABLES}

# "x ← xv con el peso mayor"
# Fija todas las variables excepto var, y elige el v que maximiza weight(xv)
# Corresponde a actualizar el nodo Xi en el diagrama (nodo sombreado).
def conditional_mode(var: str, assignment: dict) -> tuple[str, int]:
    best_server = assignment[var]
    best_weight = assignment_weight(assignment)
```

```

for server in SERVERS:
    if server == assignment[var]:
        continue
    xv      = dict(assignment)
    xv[var] = server          #  $xv = x \cup \{Xi:v\}$ 
    w       = assignment_weight(xv)
    if w > best_weight:      #  $x \leftarrow xv$  con el peso mayor
        best_weight = w
        best_server = server

return best_server, best_weight

# Equivale a recorrer los nodos  $X1 \rightarrow X2 \rightarrow X3$  del diagrama (izq-der).
# Si ningún  $Xi$  cambia en el sweep  $\rightarrow$  convergencia (óptimo local alcanzado).
def icm_sweep(assignment: dict) -> tuple[dict, bool, list[int]]:
    changed      = False
    viol_trace   = []
    for var in VARIABLES:          #  $i = 1, \dots, n$ 
        best_server, _ = conditional_mode(var, assignment)
        if best_server != assignment[var]:
            assignment[var] = best_server          # actualizar  $Xi$  en lugar
            changed         = True
        viol_trace.append(count_violations(assignment))
    return assignment, changed, viol_trace

# -- 2.3.6 Estadísticas ICM -----
@dataclass
class ICMStats:
    total_sweeps      : int = 0
    total_micro_steps : int = 0
    restarts_done     : int = 0
    found_solution    : bool = False
    initial_violations : int = 0
    final_violations  : int = 0
    violation_history  : list = field(default_factory=list)
    restart_points     : list = field(default_factory=list)

# Implementación completa del algoritmo de la diapositiva:
# Init x aleatoriamente           $\rightarrow$  random_assignment()
# Itere  $i=1, \dots, n$  hasta converger  $\rightarrow$  icm_sweep() en bucle
# Calcule pesos xv               $\rightarrow$  conditional_mode  $\rightarrow$  assignment_weight
#  $x \leftarrow xv$  con el peso mayor  $\rightarrow$  assignment[var] = best_server
# Si óptimo local sin solución  $\rightarrow$  reiniciar con nuevo x aleatorio
def icm_search(max_iter=50, max_restarts=15, seed=None, verbose=True) -> tuple[Optional[dict], ICMStats]:
    stats = ICMStats()
    rng   = random.Random(seed)
    g_sw  = 0

    for restart in range(max_restarts + 1):
        stats.restarts_done = restart

        # "Init x a una asignación completa aleatoria"

```

```

assignment = random_assignment(seed=rng.randint(0, 10**6))
init_viol = count_violations(assignment)

if restart == 0:
    stats.initial_violations = init_viol

if verbose:
    bd = violations_breakdown(assignment)
    print(f" {'='*50}")
    print(f" Reinicio #{restart} - violaciones iniciales: {init_viol}")
    if bd["anti_affinity"]: print(f" Anti-afinidad : {bd['anti_af")
    if bd["capacity"]: print(f" Capacidad : {bd['capacit

stats.violation_history.append(init_viol)
stats.restart_points.append(g_sw)

for sweep_i in range(max_iter):
    stats.total_sweeps += 1
    stats.total_micro_steps += len(VARIABLES)
    g_sw += 1

    # "Itere por i=1,...,n: x ← xv con el peso mayor"
    assignment, changed, _ = icm_sweep(assignment)
    current_viol = count_violations(assignment)
    stats.violation_history.append(current_viol)

    if verbose:
        icon = "v" if changed else "."
        print(f" Sweep {sweep_i+1:3d} {icon} viol={current_viol}")

    # weight(x) = 0 → sin violaciones → solución encontrada
    if current_viol == 0:
        stats.found_solution = True
        stats.final_violations = 0
        return assignment, stats

    # Sin cambios → convergencia a óptimo local → reiniciar
    if not changed:
        if verbose: print(f" [WARN] Óptimo local → Reinicio #{res)
        break
    else:
        if verbose: print(f" [TIME] max_iter={max_iter} alcanzado → R

stats.final_violations = count_violations(assignment)
return None, stats

```

Ejecución (Task 2.3)

ICM arranca con una asignación completa aleatoria y mejora variable por variable. Es normal ver varios reinicios hasta llegar a violaciones 0.

Observaciones:

- Cómo bajan las violaciones en cada sweep.

- Si llega a 0, la asignación final y la validación por restricciones.
- El resumen de robustez (30 corridas) para ver qué tan seguido converge.

```
In [ ]: SHOW_TRACE = False

MAX_ITER, MAX_RESTARTS, SEED = 50, 15, 42

t0 = time.perf_counter()
icm_sol, icm_st = icm_search(max_iter=MAX_ITER, max_restarts=MAX_RESTARTS,
icm_elapsed = (time.perf_counter() - t0) * 1000

if icm_sol:
    server_map = {s: [] for s in SERVERS}
    for micro, srv in sorted(icm_sol.items()):
        server_map[srv].append(micro)

    print("Distribución final:")
    for srv, micros in server_map.items():
        bar = "#"*len(micros) + "."*(MAX_CAPACITY - len(micros))
        print(f" {srv} [{len(micros)}/{MAX_CAPACITY}] {bar} -> {'', '.jo

    print(f"\nAsignación: {dict(sorted(icm_sol.items()))}\n")

    valid = True
    for a, b in ANTI_AFFINITY_PAIRS:
        ok = icm_sol[a] != icm_sol[b]
        if not ok:
            valid = False
        print(f" {'[OK]'} if ok else '[FAIL]'} {a}={icm_sol[a]} != {b}={icm

    for s in SERVERS:
        micros = [v for v in VARIABLES if icm_sol[v] == s]
        ok = len(micros) <= MAX_CAPACITY
        if not ok:
            valid = False
        print(f" {'[OK]'} if ok else '[FAIL]'} {s}: {len(micros)} microserv

    print(f"\nVálida: {'Sí' if valid else 'No'}")
    print(f"Violaciones iniciales : {icm_st.initial_violations}")
    print(f"Reinicios realizados : {icm_st.restarts_done}")
    print(f"Sweeps totales : {icm_st.total_sweeps}")
    print(f"Evaluaciones (micro) : {icm_st.total_micro_steps}")
    print(f"Tiempo (ms) : {icm_elapsed:.4f}")
else:
    print(f"No se encontró solución en {MAX_RESTARTS} reinicios.")

# Robustez (30 ejecuciones)
succ, tot_sw, tot_rs = 0, 0, 0
for run in range(30):
    sol, st = icm_search(max_iter=50, max_restarts=15, seed=run*17+3, verbos
    if sol:
        succ += 1
        tot_sw += st.total_sweeps
        tot_rs += st.restarts_done
print(f"\nRobustez (30 corridas): éxitos={succ}/30 | sweeps_prom={tot_sw/30:
```

Distribución final:

```
S1 [3/3] ### -> M2, M4, M5
S2 [3/3] ### -> M1, M3, M7
S3 [2/3] ##. -> M6, M8
```

Asignación: {'M1': 'S2', 'M2': 'S1', 'M3': 'S2', 'M4': 'S1', 'M5': 'S1', 'M6': 'S3', 'M7': 'S2', 'M8': 'S3'}

```
[OK] M1=S2 != M2=S1
[OK] M3=S2 != M4=S1
[OK] M5=S1 != M6=S3
[OK] M1=S2 != M5=S1
[OK] S1: 3 microservicios ['M2', 'M4', 'M5']
[OK] S2: 3 microservicios ['M1', 'M3', 'M7']
[OK] S3: 2 microservicios ['M6', 'M8']
```

Válida: Sí

```
Violaciones iniciales : 4
Reinicios realizados  : 0
Sweeps totales        : 1
Evaluaciones (micro)  : 8
Tiempo (ms)           : 0.3036
```

Robustez (30 corridas): éxitos=30/30 | sweeps_prom=1.0 | reinicios_prom=0.0

Resultados

Con la semilla fija, ICM llegó a una solución válida en pocos pasos y sin reinicios en la corrida mostrada. El bloque de robustez resume si esa facilidad se mantiene al cambiar la semilla. Con esto podemos ver que ICM es bastante robusto para este CSP, encontrando solución en la mayoría de las corridas sin necesidad de muchos reinicios. Sin embargo, la presencia de óptimos locales puede hacer que en algunos casos se necesiten varios reinicios para escapar y encontrar una solución válida.

Task 2.4 — Benchmarking y Conclusiones

Comparación empírica de los tres algoritmos resolviendo el **mismo CSP**:

- **Muestras:** 50 ejecuciones independientes por algoritmo (distintas semillas para ICM y Beam Search).
- **Métricas:** solución encontrada, tiempo de ejecución, operaciones realizadas.
- **Configuración:** Backtracking (MRV + FC), Beam Search K=3, ICM (max_iter=50, max_restarts=15).

Ejecución (Task 2.4)

Esta parte corre varias veces cada algoritmo sobre el mismo CSP y resume: tasa de éxito, tiempo y cantidad de operaciones.

Tip: si Beam o ICM fallan en algunas corridas, es normal (son heurísticos); por eso medimos varias ejecuciones.

```
In [ ]: N_RUNS = 50

# --- 1. Backtracking -----
bt_times, bt_nodes_list, bt_found = [], [], 0
for _ in range(N_RUNS):
    _d = {v: list(SERVERS) for v in VARIABLES}
    _st = BT_Stats()
    t0 = time.perf_counter()
    _sol = backtrack({}, _d, _st)
    bt_times.append((time.perf_counter() - t0) * 1000)
    bt_nodes_list.append(_st.nodes)
    if _sol:
        bt_found += 1

# --- 2. Beam Search K=3 -----
bs_times, bs_nodes_list, bs_found = [], [], 0
for run in range(N_RUNS):
    t0 = time.perf_counter()
    _sol, _st = beam_search(K=3, verbose=False)
    bs_times.append((time.perf_counter() - t0) * 1000)
    bs_nodes_list.append(_st.nodes_generated)
    if _sol:
        bs_found += 1

# --- 3. ICM -----
icm_times, icm_sw_list, icm_found = [], [], 0
for run in range(N_RUNS):
    t0 = time.perf_counter()
    _sol, _st = icm_search(max_iter=50, max_restarts=15, seed=run * 31 + 7,
    icm_times.append((time.perf_counter() - t0) * 1000)
    icm_sw_list.append(_st.total_sweeps)
    if _sol:
        icm_found += 1

# --- Tabla comparativa -----
print(f'''
{'='*70}
    TABLA COMPARATIVA  ({N_RUNS} ejecuciones)
{'='*70}
    {'Métrica':<28} {'Backtracking':>14} {'Beam (K=3)':>14} {'ICM':>10}
    {'-'*66}''')

rows = [
    ("Soluciones encontradas",
     f"{bt_found}/{N_RUNS} ({bt_found/N_RUNS*100:.0f}%)",
     f"{bs_found}/{N_RUNS} ({bs_found/N_RUNS*100:.0f}%)",
     f"{icm_found}/{N_RUNS} ({icm_found/N_RUNS*100:.0f}%)",
     ("Tiempo promedio (ms)",
      f"{sum(bt_times)/N_RUNS:.4f}",
```



```

        f"{sum(bs_times)/N_RUNS:.4f}",
        f"{sum(icm_times)/N_RUNS:.4f}"),
    ("Tiempo mínimo (ms)",
     f"{min(bt_times):.4f}",
     f"{min(bs_times):.4f}",
     f"{min(icm_times):.4f}"),
    ("Tiempo máximo (ms)",
     f"{max(bt_times):.4f}",
     f"{max(bs_times):.4f}",
     f"{max(icm_times):.4f}"),
    ("Nodos/ops promedio",
     f"{sum(bt_nodes_list)/N_RUNS:.1f} nodos",
     f"{sum(bs_nodes_list)/N_RUNS:.1f} nodos",
     f"{sum(icm_sw_list)/N_RUNS:.1f} sweeps"),
    ("Garantía de solución", "Sí (completo)", "No (incompleto)", "Con rei",
    ("Inicio", "Vacío", "Vacío (beam)", "Aleato
]
for label, bt_v, bs_v, icm_v in rows:
    print(f" {label:<28} {bt_v:>14} {bs_v:>14} {icm_v:>10}")
print(f" {'='*66}")

# --- Resumen de fallos -----
if bt_found < N_RUNS:
    print(f"\n[WARN] Backtracking falló en {N_RUNS - bt_found}/{N_RUNS} ejec
else:
    print(f"\n[OK] Backtracking: solución válida en {N_RUNS}/{N_RUNS}.")

if bs_found < N_RUNS:
    print(f"[WARN] Beam Search (K=3): sin solución en {N_RUNS - bs_found}/{N
else:
    print(f"[OK] Beam Search (K=3): solución válida en {N_RUNS}/{N_RUNS}.")

if icm_found < N_RUNS:
    print(f"[WARN] ICM: sin solución en {N_RUNS - icm_found}/{N_RUNS} (óptim
else:
    print(f"[OK] ICM: solución válida en {N_RUNS}/{N_RUNS}.")

bt_avg = sum(bt_times) / N_RUNS
bs_avg = sum(bs_times) / N_RUNS
icm_avg = sum(icm_times) / N_RUNS
print(f"\nPromedio (ms): BT={bt_avg:.4f} | Beam(K=3)={bs_avg:.4f} | ICM={icm

```

TABLA COMPARATIVA (50 ejecuciones)			
Métrica	Backtracking	Beam (K=3)	ICM
Soluciones encontradas	50/50 (100%)	50/50 (100%)	50/50 (100%)
Tiempo promedio (ms)	0.0557	0.2583	0.1118
Tiempo mínimo (ms)	0.0468	0.1728	0.0905
Tiempo máximo (ms)	0.1050	0.3730	0.2520
Nodos/ops promedio	8.0 nodos	66.0 nodos	1.2 sweeps
Garantía de solución	Sí (completo)	No (incompleto)	Con reinicios
Inicio	Vacío	Vacío (beam)	Aleatorio completo

[OK] Backtracking: solución válida en 50/50.
[OK] Beam Search (K=3): solución válida en 50/50.
[OK] ICM: solución válida en 50/50.

Promedio (ms): BT=0.0557 | Beam(K=3)=0.2583 | ICM=0.1118

ANALISIS EMPIRICO

Los tres algoritmos resolvieron el mismo CSP de 8 variables y 3 servidores con restricciones de capacidad y anti-afinidad.

Exactitud:

- Backtracking (completo + FC) encontro solucion en el 100% de los casos, confirmando su garantia teorica de completitud.
- Beam Search (K=3) {'encontro solucion en el 100%' if bs_found == N_RUNS else f'fallo en {N_RUNS-bs_found}/{N_RUNS} casos'} de los casos. Su heuristica (minimizar violaciones + riesgo futuro) fue suficientemente guiada para este problema, aunque teoricamente sigue siendo incompleto para K pequeno.
- ICM encontro solucion en el {icm_found/N_RUNS*100:.0f}% de los casos {'sin reinicios,' if sum(icm_sw_list)/N_RUNS <= 1.5 else 'con pocos reinicios,'} validando que el paisaje de este CSP no tiene optimos locales profundos.

Velocidad:

- Backtracking : {bt_avg:.4f} ms promedio (el mas rapido, poda FC+MRV)
- Beam Search : {bs_avg:.4f} ms promedio ({sum(bs_nodes_list)/N_RUNS:.0f} nodos por ejecucion)
- ICM : {icm_avg:.4f} ms promedio ({'pocos' if sum(icm_sw_list)/N_RUNS < 3 else 'varios'}) sweeps para converger)

Conclusion:

- La teoria se cumplio empiricamente. Backtracking con Forward Checking y MRV fue el mas confiable y eficiente para este problema. Beam Search demostro ser una alternativa viable con una heuristica informada. ICM confirma que la busqueda local es rapida por ejecucion, pero requiere reinicios para garantizar completitud practica.
-